

ASSIGNMENT 2

Design of a Vector Embedding for Capability Composition

Formal Representations, Compatibility, and Compositional Reasoning

Application Domain: Online Shopping / E-Commerce

Embedding Approach: Problem-Specific Vector Representation

Technical Report

This report presents the design, implementation, and experimental evaluation of a problem-specific vector representation for formally specified states, goals, and executable capabilities.

B.Tech Computer Science and Engineering

1. Problem Definition

The objective of this assignment is to design a problem-specific vector representation for formally specified states, goals, and executable capabilities. The representation should preserve useful relationships between capabilities so that compatibility, composition, goal relevance, and operational properties can be analysed in vector space.

The application domain selected for implementation is online shopping. An e-commerce workflow contains capabilities such as adding products to a cart, checking inventory, creating an order, making payment, generating an invoice, and sending notifications. These capabilities have explicit preconditions, effects, inputs, outputs, resources, costs, and reliability values, making the domain suitable for evaluating functional composition.

The work focuses on the embedding and composition representation rather than on redesigning a planning algorithm. A vector should not merely give every capability a unique identifier; it should contain information useful for reasoning about relationships between capabilities.

2. Design Requirements

The proposed representation was designed to satisfy the following requirements:

Capability identity: different capabilities should have distinguishable representations.

State awareness: state and goal information should be representable as vectors.

Precondition-effect compatibility: an effect produced by one capability should be usable to satisfy a precondition of another.

Input-output compatibility: capability inputs and outputs should be part of the representation.

Similarity versus composability: high vector similarity alone must not imply that two capabilities can be executed consecutively.

Composition: vectors of compatible capabilities should be combinable into a composite capability representation.

Goal relevance: capabilities useful for achieving the selected goal should be distinguishable from irrelevant capabilities.

Operational properties: cost, reliability, and availability should be represented.

3. Related Embedding Approaches

Word2Vec is relevant as background motivation because it demonstrates how discrete objects can be represented as vectors in a continuous space and how relationships can be reflected through vector similarity. However, word embeddings are not sufficient by themselves for the present problem because capability composition depends on structured functional relationships such as preconditions, effects, inputs, outputs, and resources.

Therefore, this project uses a custom, problem-specific representation rather than directly applying a general-purpose pretrained embedding model. The representation is intentionally interpretable: tokens derived from structured capability and state descriptions contribute to the vector, while operational attributes are appended as numerical features.

4. Proposed Representation

The application is described using the formal model $A = (S, C, SI, G, R, K)$, where S is the state space, C is the set of capabilities, SI is the initial state, G is the goal, R is the set of resources, and K is the set of constraints.

4.1 Initial State

User.authenticated = True

```

User.role = CUSTOMER
Cart.exists = True
Cart.item_count = 3
Order.exists = False
Order.status = NONE
Payment.status = NOT_STARTED
Inventory.available = True
Invoice.exists = False
Notification.sent = False

```

4.2 Goal State

```

Order.exists = True
Payment.status = SUCCESS
Invoice.exists = True
Notification.sent = True

```

4.3 Capability Set

Capability	Type	Main effect / purpose
AddToCart	FUNCTION	Adds a product and increases cart contents
CheckInventory	DATABASE	Checks product availability
CreateOrder	API	Creates an order from a non-empty cart
MakePayment	SERVICE	Completes payment for a created order
GenerateInvoice	FUNCTION	Creates an invoice after successful payment
SendNotification	MESSAGE	Sends a notification after successful payment
CancelCart	FUNCTION	Cancels an existing cart when no order exists
UpdateProfile	GUI	Updates the authenticated user's profile

4.4 Vector Construction

A vocabulary is built from the structured text describing the capabilities, initial state, and goal. Each item is represented by a normalized token-frequency vector. For capabilities, three operational features are appended: cost, reliability, and availability.

Thus, a capability vector has the conceptual form:

$$\phi(C_{\blacksquare}) = [\text{text_features}(C_{\blacksquare}), \text{cost}(C_{\blacksquare}), \text{reliability}(C_{\blacksquare}), \text{availability}(C_{\blacksquare})]$$

State and goal vectors are produced using the same text-vector mechanism, without the capability-specific operational feature block.

5. Mathematical Formulation

Let $V = \{v_1, v_2, \dots, v_n\}$ be the vocabulary extracted from the structured descriptions. For an object x , the text representation is a normalized token vector:

$$t(x) = [f_{v_1}(x), f_{v_2}(x), \dots, f_{v_n}(x)] / \sqrt{[f_{v_1}(x), \dots, f_{v_n}(x)]^T [f_{v_1}(x), \dots, f_{v_n}(x)]}$$

where $f(x)$ is the frequency of vocabulary token v in the description of x . For a capability C , the final representation is:

$$\phi(C_{\blacksquare}) = [t(C_{\blacksquare}), q_{\blacksquare}]$$

where q contains the selected operational attributes. In this implementation, $q = [\text{cost}, \text{reliability}, \text{availability}]$.

Similarity is measured using cosine similarity:

$$\text{sim}(x, y) = (\phi(x) \cdot \phi(y)) / (||\phi(x)|| \cdot ||\phi(y)||)$$

Compatibility is not defined solely by similarity. For two capabilities C and C' , C is considered compatible when the effects of C satisfy the state preconditions required by C' .

Composition is implemented using the component-wise mean of the vectors in a valid capability chain:

$$\phi(C \blacksquare C \blacksquare \dots \blacksquare C) = (1/k) \sum \phi(C)$$

The mean operation provides a simple composite representation, while the validity of the chain is determined separately using the functional compatibility rules.

6. Capability Composition Model

Each capability is represented using its type, inputs, outputs, preconditions, effects, constraints, resources, cost, reliability, availability, and execution mechanism.

Capability	Inputs / Outputs	Preconditions	Effects
CreateOrder	cart_id order_id	Cart exists; cart item count > 0; inventory available	Order exists; status CREATED; cart locked
MakePayment	order_id, amount transaction_id	Order exists; status CREATED	Payment status SUCCESS
GenerateInvoice	order_id, transaction_id payment_id	Payment status SUCCESS	Invoice exists
SendNotification	user_id, message notification_id	Payment status SUCCESS	Notification sent
CancelCart	cart_id status	Cart exists; Order does not exist	Cart cancelled; cart removed

The main valid composition chain evaluated in the project is:

CreateOrder → MakePayment → GenerateInvoice

The chain is valid because CreateOrder establishes the created-order state required by MakePayment, and MakePayment establishes successful payment required by GenerateInvoice.

An important distinction demonstrated by the experiments is that vector similarity and functional composability are different concepts. Two capabilities may have similar representations while still being incompatible because their state conditions do not match.

7. Implementation

The implementation is written in Python. The main program constructs the vocabulary, encodes states, goals, and capabilities, computes cosine similarity, checks compatibility using precondition-effect relationships, performs vector composition, executes the five required experiments, and saves the experimental dataset to dataset.json.

Main functions implemented:

```

encode_state(state)
encode_goal(goal)
encode_capability(capability)
similarity(x, y)
compatibility(c1, c2)
compose(capabilities)

```

The final execution produced an embedding dimension of 64 for the configured dataset and completed all five experiments successfully.

8. Experimental Methodology

Five experiments were conducted according to the assignment requirements.

Experiment	Purpose
1. Capability Compatibility	Test compatible and incompatible capability pairs.
2. Capability Composition	Construct and compare a multi-capability composite vector.
3. Alternative Implementations	Compare API, database, and GUI implementations of the same functionality.
4. Irrelevant Capability	Compare goal similarity for relevant and irrelevant capabilities.
5. Operational Attributes	Demonstrate cost, reliability, and availability features.

9. Results

9.1 Experiment 1: Capability Compatibility

Pair	Compatible?	Similarity
CreateOrder MakePayment	True	0.733
CreateOrder CancelCart	False	0.730
MakePayment GenerateInvoice	True	0.707
MakePayment SendNotification	True	0.695

The key observation is that CreateOrder CancelCart has a relatively high similarity value of 0.730 but is still incompatible. This directly demonstrates that similarity cannot be used as a replacement for functional compatibility.

9.2 Experiment 2: Capability Composition

Relationship	Similarity
Composite vs CreateOrder	0.902
Composite vs MakePayment	0.906
Composite vs GenerateInvoice	0.892

The evaluated composite capability is CompletePurchase = CreateOrder MakePayment GenerateInvoice. The composite vector remains close to the individual capability vectors, with similarities above 0.89 for all three components.

9.3 Experiment 3: Alternative Implementations

Pair	Similarity
CreateOrder_API vs CreateOrder_DATABASE	0.471
CreateOrder_API vs CreateOrder_GUI	0.680
CreateOrder_DATABASE vs CreateOrder_GUI	0.577

The results show that different execution mechanisms for related functionality can be represented in the same vector space while retaining measurable similarity.

9.4 Experiment 4: Irrelevant Capability

Capability	Relevance	Goal Similarity
CreateOrder	Relevant	0.115
MakePayment	Relevant	0.258
GenerateInvoice	Relevant	0.277
SendNotification	Relevant	0.236

Capability	Relevance	Goal Similarity
UpdateProfile	Irrelevant	0.000
CancelCart	Irrelevant	0.000

The selected relevant capabilities have non-zero similarity to the goal, while the two selected irrelevant capabilities have zero similarity in the observed representation.

9.5 Experiment 5: Operational Attributes

Capability	Cost	Reliability	Availability
AddToCart	0.000	0.990	1.000
CheckInventory	0.000	0.995	1.000
CreateOrder	0.010	0.980	1.000
MakePayment	0.500	0.970	1.000
GenerateInvoice	0.010	0.990	1.000
SendNotification	0.020	0.950	1.000
CancelCart	0.000	0.990	1.000
UpdateProfile	0.000	0.990	1.000

10. Analysis

The experiments demonstrate that the proposed representation can encode the structured information needed for the selected e-commerce capability-composition problem.

Compatibility: the explicit precondition-effect check correctly distinguishes the required compatible and incompatible pairs.

Similarity is not composability: the CreateOrder – CancelCart example shows why vector similarity must be interpreted separately from functional validity.

Composition: the composite vector remains close to its component capabilities, indicating that the mean operation produces a representation related to the capabilities participating in the chain.

Alternative implementations: API, database, and GUI implementations can occupy the same vector space and can be compared using cosine similarity.

Goal relevance: relevant capabilities have non-zero observed goal similarity, while selected irrelevant capabilities have zero similarity.

Operational information: cost, reliability, and availability are explicitly incorporated into capability representations.

Overall, the implementation provides a simple and interpretable baseline for vector-based capability composition. It is intentionally designed around the formal structure of the application rather than relying on a generic language embedding.

11. Limitations

The text representation is based on normalized token frequencies and therefore does not capture the deeper semantic relationships available in learned neural embeddings.

The current composition operation is a component-wise mean, so it does not preserve the order of capabilities inside a chain.

The compatibility test is based on exact state precondition/effect matching and therefore does not cover more complex logical expressions or inferred state transitions.

Operational attributes are represented numerically but are not themselves optimized for multi-objective planning.

The experimental dataset is small and domain-specific, so larger datasets and additional domains would be needed to assess generalization.

Input-output information is included in the capability representation, while the main compatibility experiment focuses primarily on state precondition-effect satisfaction.

12. Conclusion

This project designed, implemented, and evaluated a problem-specific vector representation for formally specified states, goals, and executable capabilities in an online-shopping domain. The representation combines structured text features with operational attributes and supports state encoding, goal encoding, capability encoding, similarity measurement, compatibility checking, and capability composition.

The experimental results demonstrate the required functional behaviour: compatible capabilities can be identified, incompatible capabilities can be distinguished even when their vector similarity is relatively high, capability chains can be composed, alternative implementations can be compared, irrelevant capabilities can be separated from goal-related capabilities, and operational properties can be included in the representation.

The work therefore provides an interpretable baseline for representing capability relationships in vector space. Future improvements could use learned embeddings, richer logical compatibility models, order-sensitive composition, and larger experimental datasets.

Appendix A Capability Operational Data

The following values are the operational attributes used by the implementation and saved in the generated dataset.json file.

Capability	Cost	Reliability	Availability
AddToCart	0.000	0.990	1.000
CheckInventory	0.000	0.995	1.000
CreateOrder	0.010	0.980	1.000
MakePayment	0.500	0.970	1.000
GenerateInvoice	0.010	0.990	1.000
SendNotification	0.020	0.950	1.000
CancelCart	0.000	0.990	1.000
UpdateProfile	0.000	0.990	1.000

Appendix B Submission Files

main.py Python implementation of the embedding and experiments.

dataset.json Generated experimental dataset.

README.md Project description and instructions for running the implementation.

Assignment_2_Final_Report.pdf This technical report.

Final project workflow: place these files in the GitHub repository, run main.py once to verify the output and dataset, check that all files are visible in the repository, and then submit the repository link.